GFX2D：基于 RISC-V 与 FPGA 异构架构的 2D 图形渲染加速引擎

摘　要

本作品在 Ti60F225 FPGA 上设计并实现了软硬协同的 2D 图形渲染加速系统 GFX2D：硬件为自研 BitBlt（位块传输）渲染引擎，软件为运行于 RISC-V 软核的自研 C 驱动、软件执行器与弹幕游戏，真实游戏从纯软件 2.5 FPS 加速到稳定 60 FPS（约 24 倍）。硬件引擎承担命令解析、图像块拷贝、纯色填充、Alpha 混合与色键去背，其中 Alpha 混合由 12 个 DSP 混合核完成，引擎作为 AXI4 主机直接访问 DDR3 显存，扫描输出经 HDMI（TMDS）以 640×480@60 Hz 显示；软件部分运行于易灵思官方的 Sapphire RISC-V 软核 SoC，C 驱动、与硬件同规格的软件执行器及弹幕射击游戏 MYRIAD 构成完整软件栈，逐帧发布 32 字节渲染命令驱动引擎执行。直通、条带、硬件分桶三种执行模式在同一比特流上切换，共用一套命令语义与像素流水线。上述设计经板上实测验证，在开启 Alpha 混合与色键的条件下，以真实游戏负载自动搜索测得稳定 60 FPS 的同屏 Sprite（精灵）极限 N_{max} 为 2187 颗（8×8），以 3600 场逐场零掉帧确认；参数缓冲后端在同板同程序对照下再提高 12.3%，每帧读突发减少 70%、写突发减少 92%；混合方程舍入经 8,421,376 组穷举一致，板上 9,208 帧逐帧一致；整机占用 60064 个 XLR（98.8%）、172 块片上 RAM（67.2%）、23 个 DSP（14.4%），核心时钟 setup/hold 余量 +0.104/+0.022 ns，无时序违例。**系统完整实现赛题 7 项基础要求与 3 项高阶要求（见表 1），并给出五项创新点（见 1.5 节）。**

关键词：2D 图形加速；FPGA；RISC-V；软硬件协同；DDR3

赛题要求完成情况对照

表 1　赛题要求完成情况对照

类别	赛题要求	完成情况
基础要求	解析渲染指令（操作类型、源/目的地址、宽、高）	✅ 已完成，命令解析 159 条语义向量全过
	图像块拷贝与纯色填充	✅ 已完成，同帧 CRC 验证逐像素等价
	HDMI 输出不低于 640×480@60	✅ 已完成，displayTest 实测 59.9 FPS
	总线直接访问内存，突发与 FIFO	✅ 已完成，AXI4 直访，突发 256 拍实测，欠载 0
	C 驱动配置寄存器	✅ 已完成，C 驱动全链路并上板运行
	双缓冲防撕裂	✅ 已完成，场消隐锁存切换，SWAP_COUNT 对账
	屏幕实时对比软/硬帧率	✅ 已完成，同屏对比，60 FPS 对 2.5 FPS（24 倍）
高阶要求	硬件 Alpha 混合（DSP）	✅ 已完成，12 个 DSP 混合核，穷举校验一致
	Color Keying	✅ 已完成，整字色键，KEY_SKIP 计数器对账
	高负载游戏 Demo，60 FPS 下冲同屏 Sprite 极限（8×8）	✅ 已完成，自动搜索，N_{max} 2187 颗，零掉帧
创新点	硬件分桶渲染	✅ 已完成，同板同程序对照 N_{max} 提升 12.3%
	片上纹理（读方向）	✅ 已完成，高频小素材读事务降为零
	条带缓冲（写方向）	✅ 已完成，弹幕随机小写合并为条带连续写

第一部分　作品概述

1.1 功能与特性

本作品是一套完整的 2D 渲染加速系统，包含硬件渲染引擎、显示输出链路、板上软件平台和一个真实可玩的弹幕射击游戏。

每帧的渲染流程为：RISC-V 处理器运行游戏逻辑，把绘制需求编码为 32 字节渲染命令写入 DDR3 命令表，写 DOORBELL 触发引擎取指；引擎以每周期 4 像素的吞吐完成像素搬运、Alpha 混合与色键去背，结果写入帧缓冲；扫描输出按场同步取数，经双缓冲与 TMDS 编码后以 HDMI 输出画面。

引擎保留直通、条带与硬件分桶三种执行模式，可在同一比特流上运行时切换：直通以 16 KiB 片上纹理承接高频小素材的读，条带以片上行缓冲把弹幕的随机写合并为条带连续写，硬件分桶把命令整理与分桶计算搬进硬件，CPU 每帧只发布一份原始绘制清单，各条带的紧凑引用表（参数缓冲）由引擎生成，跨条带 Sprite 不再复制命令。

三个演示模式覆盖赛题全部要求：基础演示（方块运动与大图切换，屏幕同屏显示软/硬件两路帧率）、可玩游戏（默认开启 Alpha 混合与色键）、压力测试（自动搜索稳定 60 FPS 下的同屏 Sprite 数量极限）。

1.2 应用领域

本系统面向以图层合成为主的 2D 嵌入式图形场景：仪表与设备的图形界面、图标与半透明面板的叠加显示，以及 Sprite 类 2D 游戏。这些场景由块拷贝、纯色填充、Alpha 混合与色键等基本操作构成，不需要 3D 变换与抗锯齿。渲染引擎的命令接口带版本号与能力寄存器，混合方程以表格方式扩展；引擎对外仅暴露寄存器与命令表两类接口，也可作为图形子系统集成进更大的 SoC 方案。

1.3 主要技术特点

本作品的技术特点体现在三方面。其一，全链路自研：除厂商提供的 SoC 软核、SDK 生成物与开发工具外，渲染引擎、显示链路、C 驱动、软件执行器、游戏与素材工具链均为自行设计，各层接口契约清晰，问题可定位到任意一层。其二，可观测与可对账：引擎内建九个性能计数器并附带恒等约束（如读 beat 数等于各读突发长度之和），帧率以场数减切换数客观定义，错误可定位到具体命令的字段，性能结论均绑定串口日志与产物指纹，可回溯到具体版本。其三，软硬同规格双实现：软件执行器与硬件流水线实现同一份逐像素规格，互为差分测试的对照，混合方程穷举校验与万张级命令表差分因此成为常规回归手段。

1.4 主要性能指标

主要指标见表 2，全部为板上实测。

表 2　主要性能指标（实测）

指标	实测值
显示与渲染	640×480 @ 60 Hz，HDMI（TMDS），双缓冲无撕裂；每周期 4 像素，12 个 DSP 混合核 @ 100 MHz
帧率（普通游戏短测）	硬件后端稳定 60 FPS（首秒预热），纯软件约 2.5 FPS，加速约 24 倍
N_{max}（硬件分桶后端，8×8 Sprite）	2187 颗，3600 场零掉帧（同板同程序对照 +12.3%，见表 5）
负载能力（过载档，约 2030 活动弹）	30.6 FPS，FIFO 零欠载
整机资源	60,064 XLR（98.8%）、172 块片上 RAM（67.2%）、23 个 DSP（14.4%）
时序	core_clk setup 余量 +0.104 ns，hold +0.022 ns，无违例

1.5 主要创新点

（1）硬件分桶渲染：分桶计算与跨条带命令复制移入引擎，N_{max} 同板同程序对照提升 12.3%（2.2.5 节）。

（2）片上纹理：16 KiB 片上存储承接高频小素材，其 DDR3 读事务降为零（2.2.5 节）。

（3）条带缓冲：弹幕的随机小写合并为条带连续写（2.2.5 节）。

（4）命令 ABI 与十步错误模型：错误可定位到具体命令与字段（2.2.4 节）。

（5）以实测反推架构：先实测控制器事务模型，数据布局与访存形态据此而定（2.2.2 节）。

1.6 设计流程

设计遵循实测先行、判据闭环的流程：开工前以探针主机实测 DDR3 控制器事务模型与多路器仲裁行为，确定访存形态与端口分配；随后划定存储地图与像素格式，完成引擎微架构设计与 RTL 实现；软件执行器与硬件同步开发，作为金标准支撑逐层对账；最后经综合时序收敛、比特流上板，以自动 N_{max} 搜索与 3600 场零掉帧确认收口。条带与分桶两种执行模式按同一流程递进叠加。

1.7 相关工作与本作品的定位

2D 图形的硬件加速有两条主线。第一条是合成语义：Porter 与 Duff 把 Alpha 合成抽象为一组算子 [8]，此后硬件混合单元的语义基本建立在这套代数上，本作品的六种混合方程与预乘主路径正是它的直接实现。第二条是固定功能 2D 引擎：面向 MCU 的 DMA2D 类加速器以寄存器级命令完成块拷贝、格式转换与两图层混合 [10]；FPGA 上也有面向 Sprite 的专用引擎，例如用高层次综合与片上存储实现多 Sprite 处理的硬件引擎 [13]，以及 2D 图形系统的软

硬协同设计 [14]。这类工作把逐像素搬运与混合从处理器移入硬件，命令通常由处理器逐条写入、执行模式单一。

当像素通路不再是最紧的瓶颈，限制随之转移到访存。Rixner 等人的访存调度研究表明，行缓冲命中与突发组织对有效带宽的影响可以超过峰值带宽本身 [12]；Molnar 等人的排序分类则指出，按空间局部性重排访问可以把随机访问转成连续访问 [11]，移动端的图块化渲染沿用了同一思路 [16]。1.5 节前三项创新点正是把这两条结论落到 2D 弹幕负载上的结果。

与上述工作相比，本作品的定位在三点：命令以 32 字节定长表提交、由引擎自取，业界引擎的命令通常由处理器逐条写入；同一比特流上提供直通、条带与硬件分桶三种执行模式，可按负载切换并同板对照，业界引擎执行模式单一；为读、写、构造三个方向各建一级片上缓冲（16 KiB 片上纹理、640×16 行条带缓冲与参数缓冲），传统 2D 引擎没有这一存储层级。

第二部分　系统组成及功能说明

2.1 整体介绍

系统由一颗软核处理器、三个自研硬件模块和标准存储/显示外设组成，全部装载于赛题指定的 Ti60F225 开发板。如图 1 所示，Sapphire SoC（VexRiscv RV32IM @ 100 MHz）运行游戏与驱动；自研的 GFX2D 渲染引擎与扫描输出模块 display 是 DDR3 的另外两个总线主机，三者经厂商 AXI 多路器 Axi_Mux（三进一出，128 位）共享 DDR3 控制器；axi_mux_guard 为处理器通路提供防饥饿保护。引擎与显示模块的控制寄存器、状态寄存器与性能计数器挂在 APB（Advanced Peripheral Bus）总线（0xF810_0000 起），完成与错误经两条中断线上报处理器；图中虚线表示 APB 控制通路，点线表示中断线。

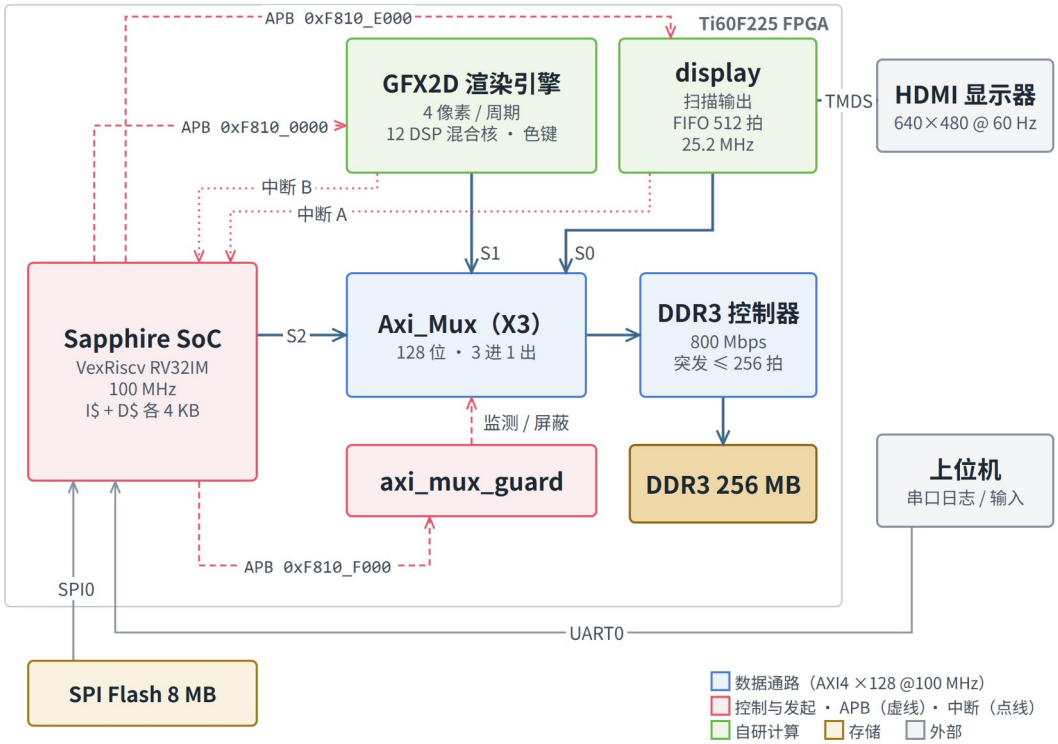


图 1　系统总体架构

三个总线主机的分工是刻意设计的：扫描输出是实时流，每个场必须取完 300 笔行读，故独占只读端口 0 且每方向只保留一笔在途事务；渲染引擎是吞吐型负载，接端口 1，读写各一笔在途，既保持流水满载，又避免处理器通路陷入饥饿；处理器接端口 2，由永久固化的 guard 模块监测地址等待超龄并暂时屏蔽引擎的新授权。这一分配在整机仿真中验证：引擎遵守一笔在途契约时，处理器地址等待不超过 414 个周期，画面扫描从不缺数。图中 S0/S1/S2 即上述 0/1/2 号端口，分别接 display、渲染引擎与 Sapphire SoC。

一帧的完整数据流如下。处理器在帧 N 更新游戏状态（定点运算、碰撞检测、实体池压缩），生成约两千条 32 字节命令写入命令表，写 LIST_BASE 与 LIST_COUNT 后写入 DOORBELL 触发取指；引擎取指、检查、裁剪，把每个矩形展开成行段，经像素流水线读写 DDR3 完成混合；渲染完成触发中断 B，处理器请求换页，扫描输出在下一个场消隐期把前台指向新帧；HDMI 输出该帧。CPU 生成命令与引擎渲染上一帧命令同时进行，生成与渲染两段流水并行。

2.2 硬件系统介绍

2.2.1 硬件整体与存储地图

Ti60F225 具备 60.8K 逻辑单元、256 块 10 Kbit 片上 RAM 与 160 个 DSP。本设计最紧缺的资源是片上 RAM：一帧 640×480 的 32 位图像约 4.9 Mbit，超出全部片上容量，因此帧缓冲、素材与命令表都必须驻留 DDR3（256 MB，16 位颗粒 @ 800 MT/s，控制器 AXI 数据宽 128 位、地址 28 位）。芯片上的 RAM 预算优先投给两处跨时钟 FIFO 与引擎的读写缓冲，DSP 预算集中投给像素混合通路与分桶前端的地址计算。

DDR3 地址空间按各区域的写入主机、读取主机与每帧写入次数划分，如图 2 所示。程序区约 4 MB；两张各 1 MB 的命令表支持双表流水；素材区 16 MB 存放启动时从 Flash 装入的纹理图集与字库，CPU 只写一次、引擎只读；

两个 4 MB 帧缓冲构成双缓冲对；另设 8 MB 离屏层供背景预合成与调试抓帧。Flash 8 MB 恰好放满比特流（2 MB）、应用（512 KB）与素材包（4 MB），上电由 bootloader 引导应用、应用再经 SPI0 把素材装入 DDR3。

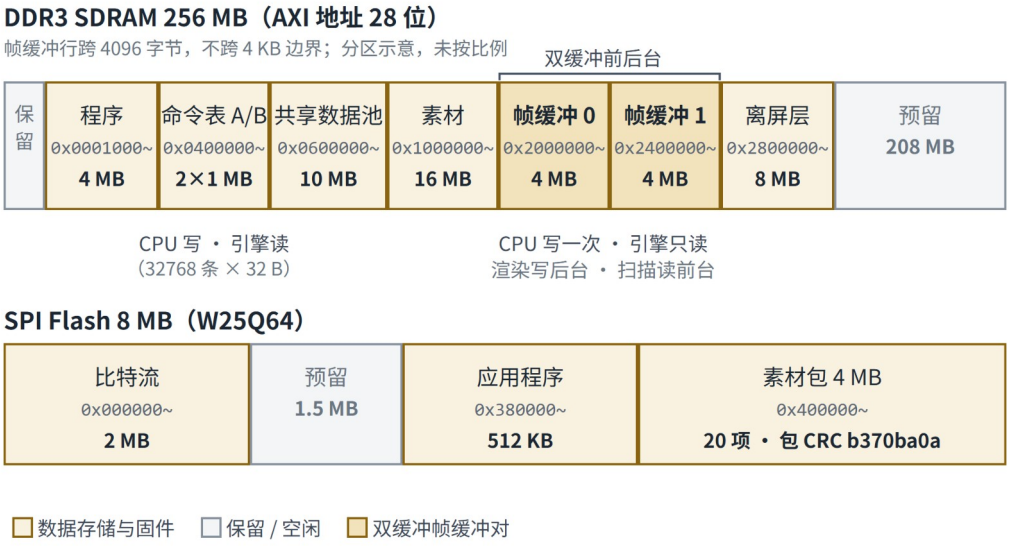


图 2　DDR3 与 SPI Flash 的地址分区

像素格式选 32 位 XRGB8888（帧缓冲）与预乘 ARGB8888（纹理），一个 128 位 AXI 拍正好装 4 个像素，搬运与混合都按 4 像素对齐。帧缓冲行跨取 4096 字节，一行 640 像素恰好一次突发、永不跨 4 KB 边界——AXI4 协议禁止突发跨越 4 KB 边界，这个约束在数据布局层面被直接消掉，硬件不再需要任何边界检查逻辑；地址计算也退化为移位。这两条设计决策都源自开工前对控制器与协议的定量实测，是本设计“预算先行”方法的第一处落地。

2.2.2 总线接入与存储仲裁

引擎作为独立主机直接访问内存，经 Axi_Mux 三端口接入。接入前用探针主机实测了 DDR3 控制器的事务模型：突发上限 256 拍，每笔事务周期数 $T(N) = t_tx + k \times N$，其中写事务固定开销 $t_tx = 7.25$ 周期、每拍增量 $k = 1.183$；读事务固定开销高达 27.41 周期、增量 1.086。也就是说，一笔只读 1 拍的事务要花 29 个周期，摊到每拍的开销是 256 拍满突发情形的约 24 倍。持续吞吐实测为每周期 0.85~0.92 拍，折合带宽远超扫描输出与整帧读写的合

计需求——带宽不是瓶颈，事务次数才是。这条实测结论决定了引擎的一切访存形态：读方向避免小块读，源与目的按行合并成长突发；写方向按行凑满 160 拍。

多路器的仲裁行为也以实测为准。直接仿真厂商多路器的授权状态机发现：三路持续请求时端口 0、1 交替而端口 2 陷入饥饿（starvation），厂商所称“优先级模式”并非严格优先级。本设计不依赖对厂商逻辑的猜测，而是用三层措施形成边界：端口 0 扫描与端口 1 引擎各自的一笔在途契约（由引擎 RTL 保证）、永久模块 axi_mux_guard 对端口 2 地址等待的老化监测（超过 64 个周期即从新授权中暂时屏蔽引擎）、以及显示 FIFO 的水位与欠载计数。板上三方并发实测：扫描 FIFO 在探针满负载读、满负载写与空闲三种负载下最低水位分别为 347 / 331 / 327 拍，触底与欠载均为零；整机接入引擎后按同一判据复测最低 335 拍，压力搜索全程最低 332 拍、欠载为零。

CPU 侧的数据缓存不含一致性机制，引擎直接写 DDR3 会与缓存副本脱节。本设计把共享区使用规则定为“读回前必须失效”，并通过机理实验标定了代价：失效指令本身全清约 67 周期、单地址约 4 周期，失效后每行首次访问重新装行约 69 周期。整帧读回约 13 ms（估算），因此逐帧 CRC（Cyclic Redundancy Check）对比作为调试与验收手段按需开启，不进入生产帧路径；游戏运行期需要引擎结果的场合一律走寄存器与中断。

2.2.3 扫描输出与双缓冲

显示链路如图 3 所示。display_fetch 按异步 FIFO 水位发起 160 拍的行读（一行 640 像素恰好 160 拍），数据经 128 位×512 深的异步 FIFO（First-In First-Out）从 100 MHz 总线域跨到 25.2 MHz 像素域；display_out 把每个拍拆成 4 个像素、处理对齐相位，并按 SRC 寄存器选择输出彩条或帧缓冲；hdmi_tx 完成 TMDS 编码。上电默认输出八色彩条，显示基线不依赖任何软件——这也是一条独立的回归判据：无论渲染侧如何改动，彩条必须始终正确。



场消隐期锁存新帧基址完成双缓冲切换（见双缓冲时序图）；上电默认输出八色彩条。

图 3　从 DDR3 帧缓冲到 HDMI 的扫描输出通路

字节序是一个容易被忽视的正确性风险：小端内存中 128 位拍内 4 个像素的字节排列与编码器输入的位段必须严格同序。本设计先经推导确认“拆像素不需要任何字节交换”，再在板上用四条横向色带实验做了端到端验证，屏幕自上而下呈现红、绿、蓝、白与推导一致。

双缓冲与场同步的时序如图 4 所示。软件写 BACK_BASE 与 SWAP_REQ 后，切换在下一个场消隐期由硬件锁存完成，请求位自清；若引擎未能在消隐前渲染完，该场保持前缓冲并被计为一次掉帧——帧率统计因此有客观定义（场数减切换数），不依赖目测。list_done 晚于消隐到达时，该场不切换；图中菱形标记即为消隐期切换点。两份命令表与双缓冲配合形成两级流水：CPU 填表 A 时引擎渲表 B，消隐时呈现 B 的结果，画面始终完整。

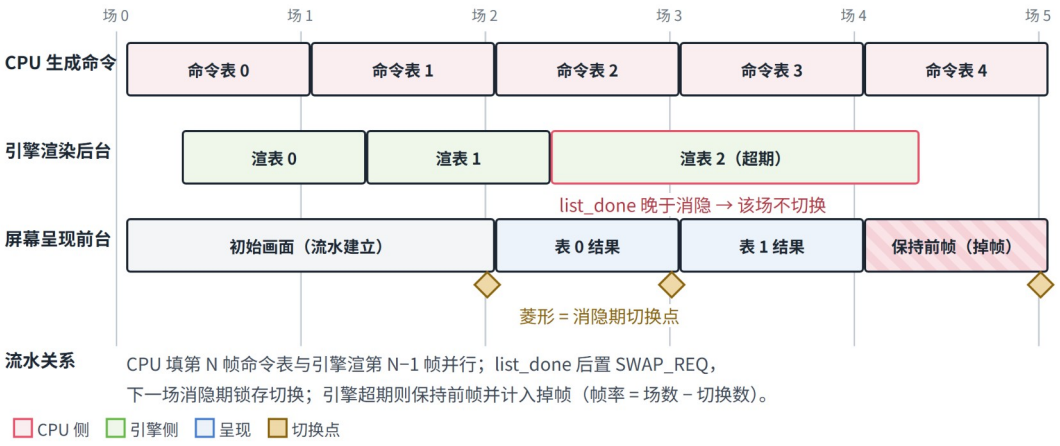


图 4　双缓冲与两级命令表流水的场同步时序

2.2.4 2D 渲染引擎

引擎是本作品的核心硬件，内部结构如图 5 所示，自上而下五个部分：APB 从机与寄存器组、命令取数、检查与解码、几何层、像素流水线，底部是 AXI4 事务层。所有“地址”经表面表间接寻址：驱动先在 SURF[0..15] 登记表面的基址、宽高、行跨、地址空间与像素格式，命令只携带表面号与坐标，硬件统一完成裁剪与越界检查。这既满足赛题“指令包含源地址、目的地址、宽度、高度”的要求（源/目的地址由表面基址与坐标换算给出），又让非法引用在进入像素通路之前就被拦截。

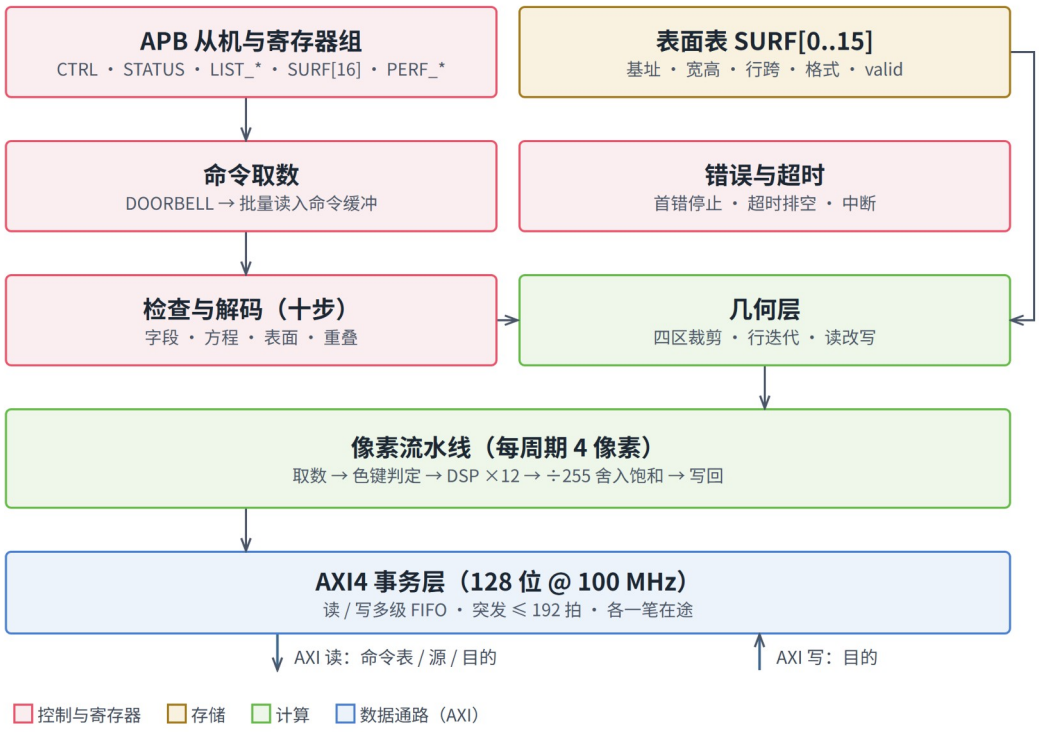


图 5　渲染引擎的分层结构

命令由 DOORBELL 寄存器受理后按 LIST_BASE 从 DDR 批量读入片上缓冲，LIST_COUNT 为硬边界，END 可提前结束。命令记录固定 32 字节 8 个字，记录长度取 2 的幂使硬件用移位即可定位第 k 条命令。操作码共五种：NOP、FILL（纯色填充）、BLIT（图像块拷贝）、SET_CLIP（设置持续生效的裁剪框）与 END（提前结束符）；w0 内嵌混合方程编号、色键使能与常数 alpha，w1~w5 携带源/目的坐标、宽高、常量源像素与色键颜色。命令提交后

由固定十步检查（操作码、字段、方程、源表面、目的表面、地址空间、格式、预乘约束、重叠）把关，首错停止：失败命令不产生任何像素副作用，错误码与出错记录索引锁存在寄存器中并触发中断，软件可精确定位到表内第几条命令的哪个字段。总线类错误（读/写响应非 OKAY、事务超时）同样进入该通路，超时后引擎排空在途事务再回到空闲，保证 DDR3 上同时运行的处理器不受牵连。

像素流水线如图 6 所示，每级处理一个 128 位拍（4 像素）：取数、色键判定、DSP 乘、加与舍入、写回。混合统一为 out = src×Fs + dst×Fd，六种方程共用同一条通路：COPY（拷贝）、ALPHA（预乘半透明，主路径）、ALPHA_STRAIGHT（非预乘对照路径，直接实现赛题公式）、ADD（加法发光）、CONST（整屏常系数）与 MUL（乘法暗化）。主路径纹理预乘 alpha 后每通道只剩一次乘法；12 个混合核（4 像素 × RGB 三通道）各占用一块 DSP，与赛题“利用内部 DSP/硬件乘法器”的要求逐字对应。除以 255 的舍入采用快速式 out = min(255, (t + (t>>8)) >> 8)，其在全定义域与 half-up 精确商逐值一致（穷举校验），中间量保留 17 位避免模截断误差。色键按源表面的 32 位存储字整字比较，命中像素不写回，实现不规则边缘 Sprite 的精确叠加；混合只作用于 RGB 三通道，写回 XRGB8888 帧缓冲时高字节强制为 0。

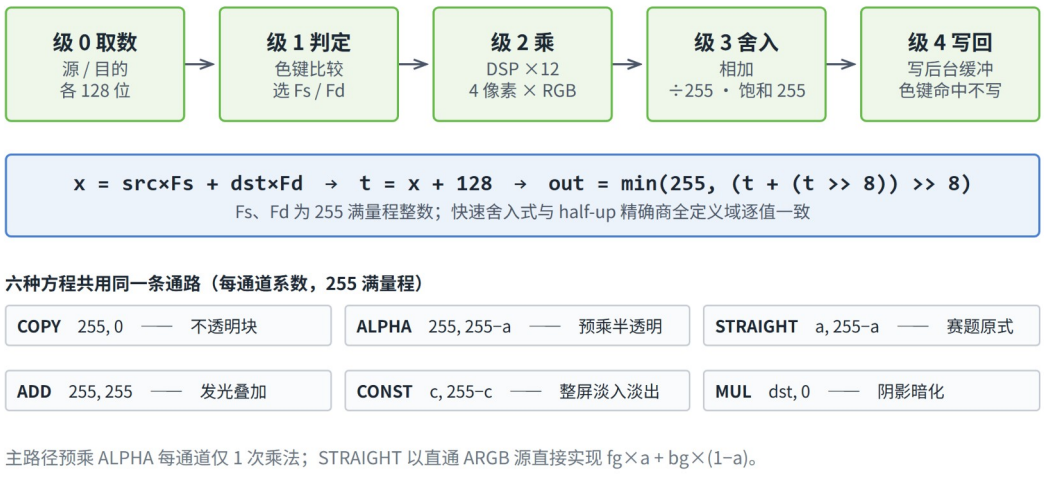


图 6　像素流水线与混合方程

引擎内建九个性能计数器（命令数、像素数、色键跳过数、读写突发数、读写 beat 数、等 DDR 周期数、忙周期数），并保证可对账约束（如读 beat 数等于各读突发长度之和）——计数器不是装饰，而是第 2.4 节验收判据的数据来源。

2.2.5 三种执行模式与片上存储

引擎保留直通、条带与硬件分桶三种执行模式，由控制寄存器的模式位运行时切换，同一比特流下各模式的性能可作同板对照。三模式共用 2.2.4 节的命令语义、十步检查与像素流水线，差别只在数据的来路与去处。

片上纹理（TEX）服务读方向。16 KiB 源纹理按四 bank 组织（实测占 16 块片上 RAM），经 APB 窗口（0xF810_4000 起 4 页）由处理器装载：游戏在每关开始时把高频小素材（标准圆弹、小圆弹与火花，合计约 10 KiB）从 DDR3 素材区抄入片上，之后处理器不再访问，只有引擎读。装载通路选 APB 而非 DMA，依据是量的对比：热集 10 KiB 每关一次、折合 2,560 次 32 位写，而每帧 1.2 MB 量级的帧面读写从来都走引擎的 AXI 主机通路，APB 只承担小而稀的寄存器与装载流量。源标记随命令同拍锁存进描述符，搬运层在描述符入口把 TEX 源引向片上读口，这类命令的 DDR3 读事务降为零。实测读事务固定开销 27.4 周期、弹幕命令以小源读为主，正是它构成直通模式的主要墙。

条带模式（BAND）服务写方向。640×16 行×32 位的目的行缓冲（四像素写使能分段组织，实测 36 块片上 RAM）按条带循环执行三相互斥的阶段：LOAD 从背景面装入该 16 行，EXEC 以 BAND 为目的执行桶内命令，FLUSH 把整条带连续写回帧面。弹幕写入由散落帧面的随机小写合并为 30 次条带连续写。命令表升级为定长条带表（ABI v2）：表头 128 字节为各桶计数数组，其后 30 个桶各 1,024 条、以 32 KiB 步长紧排，整表不足 1 MiB；桶内命令保持原绘制顺序，跨桶命令按裁剪后矩形复制进每个覆盖桶，目的坐标在桶内换算为条带局部行；SET_CLIP 在构造期吸收进矩形，条带原始表禁用该命令；

任一桶溢出则整表拒绝，错误索引携带条带维度。条带高度 16 行经参数 STRIP_H 全链贯通，支持 8~32 行二次幂对照。

硬件分桶（参数缓冲）服务构造方向，消除条带表“跨桶复制完整命令”的 CPU 成本。CPU 每帧只提交一份原始绘制与实例数组，引擎以两次遍历建立引用：第一遍按输入序完成共享检查与裁剪快照，统计每个条带的引用数；第二遍按同一顺序把引用（绘制包号与实例号的对）散写进各条带的引用区，桶内天然保持输入顺序。引用表（参数缓冲）由 512 字节头部、64 字节绘制元数据槽与 8 字节引用组成，容量以寄存器配置，超限在渲染前显式失败；双参数缓冲区按帧代次租借，一帧的全部读写完成并排空后才可复用。执行阶段每桶只遍历自己的引用，元数据与实例数组按需从 DDR3 取用。该模式复用直通与条带的几何、像素与条带通路，逐像素语义与前两种模式一致，同板同程序性能对照见第三部分 3.3.2 节。

三种模式合用后整机片上 RAM 为 172 块（67.2%），其中分桶计数、游标与半引用缓冲合计 8 块。新模式的像素语义经独立直通软件金标准、真实游戏整帧 RTL 对拍与板上同帧 CRC 交叉验证，与直通逐像素一致；整机收益的同板同程序实测对照见第三部分 3.3.2 节。

2.3 软件系统介绍

2.3.1 软件整体：分层与双目标构建

软件侧的分层如图 7 所示：应用层（游戏与判据程序）、算法层（lib/，纯 C）、板级支持层（bsp/）与厂商层（sdk/，SoC IP 生成物，只读）。分层的可见范围由编译系统的头文件搜索路径强制：算法层看不见任何平台头文件，应用层看不见 sdk，越界引用直接编译失败，并有脚本以故障注入方式验证这条性质本身。

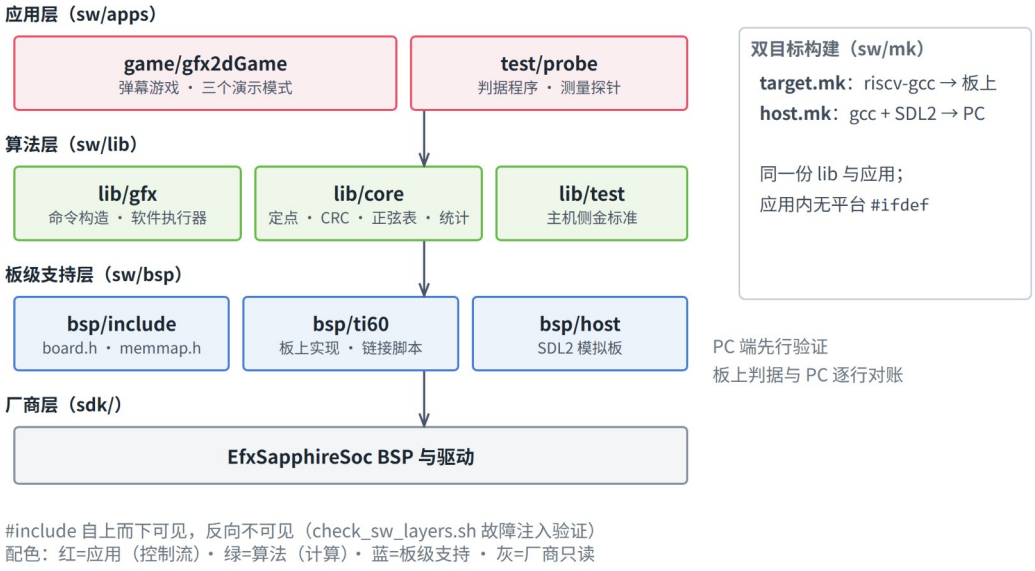


图 7　软件分层与双目标构建

这套结构带来一个关键能力：同一份算法层与应用层代码可以直接编译为 PC 程序。bsp/host 用 SDL2 提供窗口、键盘与“内存模拟的 DDR 分区”，PC 版游戏先于板上开发与调试；板级接口 board.h 在两端语义一致（场节拍、呈现、输入、串口），使“PC 上逐行对账、板上复现”成为常规操作。游戏开发、命令表构造、统计埋点都在 PC 上完成，板子只承担最终验证——这把一块开发板变成了一整个开发平台。

软硬件的分工在帧循环里一目了然：处理器负责游戏逻辑（实体池更新、弹型参数化发射、碰撞检测、关卡与规则）、命令表构造与引擎调度；引擎负责一切逐像素操作。处理器侧不做任何逐像素运算，这是“把 CPU 从像素搬运中解放出来”的赛题命题在本设计中的直接回答。

2.3.2 软件各模块

命令表构造层与软件执行器。 构造层把游戏的绘制意图翻译成命令记录：表面登记、矩形提交、裁剪管理、命令表滚动缓冲。软件执行器按与硬件完全相同的十步检查和逐像素规格解释同一份命令表，它有三重身份：PC 上的渲染后端（游戏可玩）、硬件的参照实现（金标准）、以及统计埋点宿主（逐帧像素数、按方程分布）。板上渲染后端共五种——纯软件解释器、硬件直通、硬

件片上纹理、硬件条带与硬件分桶（参数缓冲）——运行时按键循环切换，同一比特流即可完成对照，HUD（Head-Up Display）同屏显示当前后端帧率，这正是赛题要求的“实时对比”。

条带构造与多后端提交。 条带命令表由同一组组帧 API 构造：组帧器不感知条带，构造层在追加路径内部把每条命令的矩形与 30 个条带求交，跨桶复制、换算条带局部行并写入对应桶，SET_CLIP 吸收为构造期裁剪框——游戏组帧代码在直通与条带两种表上零改动，产出逐字节可对拍的两份命令表。驱动按表版本分派提交入口：直通表携带基址与条数，条带表携带基址、条带数、条带高度与背景/目的表面号，引擎侧模式位随之切换。软件执行器同样具备条带解释模式（逐桶装载影子面、执行、写回，失败桶不污染帧），作为条带路径的金标准与板上同帧对拍的参照实现。

分桶后端发布。 分桶模式下构造层进一步退化：每帧把绘制编码为一份原始绘制与实例数组（共享绘制状态加每实例 4/8 字节属性），驱动把它发布进双参数缓冲租借区、登记帧代次并写门铃，分桶、引用构建与跨带复制全部由硬件完成；一帧完成并排空后该帧的参数缓冲才回到租借池。CPU 侧保留独立的分桶参考模型，板上以逐桶计数、引用内容与同帧像素 CRC 对账；直通与条带两种提交路径保持原状，作为回退与对照。

素材流水线。 如图 8 所示，PNG 图集与开源字库（Spleen，BSD-2 许可）经打包脚本解码为预乘 ARGB 图集并附目录头，素材包 2,840,320 字节、包校验码 b370ba0a；与比特流、应用镜像合成为 Flash 镜像一次烧写。上电后 bootloader 从 0x380000 引导应用，应用经 SPI0 把素材装入 DDR3 素材区并逐项校验，最后把表面登记进引擎。装载通路对坏包显式拒绝（九项判据），素材包在新字库引入后重打包三次板上回归全过。

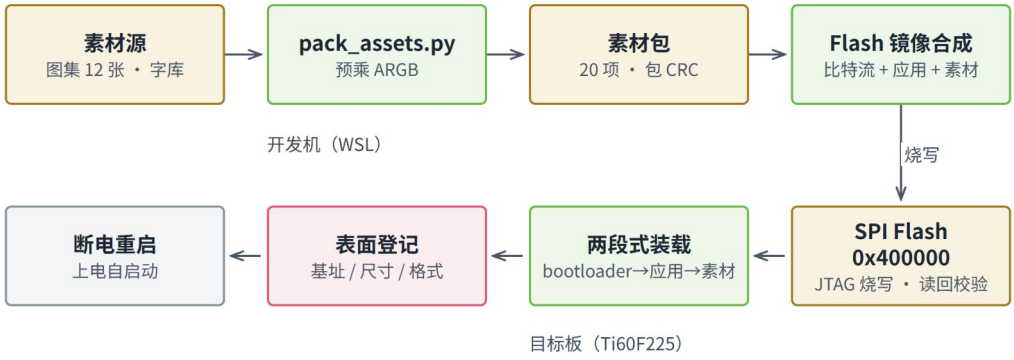


图 8　素材流水线与启动装载

游戏本体 MYRIAD。 竖版弹幕射击，左侧 400×480 游戏区、右侧 240×480 HUD（分数、残机、活动弹数、软/硬件帧率）。数值全程定点：位置与速度 Q.8 定点，角度用 16 位二进制角查 1280 项正弦表，避免无 FPU 处理器上的浮点开销。实体采用四池管理（敌弹、自弹、粒子、拾取物）与离屏立即回收，更新循环无函数指针调用、无动态分配。弹型由发射器参数方程生成（圆环、螺旋、自机狙、扇形、随机雨），关卡为“30 秒杂兵 + 四张符卡”的 Boss 战结构，终符的密度随时间上升，兼作压力测试负载。碰撞为 O(N) 圆形判定，自动炸弹与残机规则完整，输入支持板上按键与上位机经串口转发的键盘/手柄。

三个演示模式。 模式 1 基础演示：大量方块随机移动与多张大图快速切换两个场景，屏幕同屏显示软件与硬件两路帧率；模式 2 游戏：完整可玩关卡，默认 Alpha + 色键；模式 3 压力测试：以终符为负载，自动执行“N_{max} 搜索”，把结果（含搜索轨迹）实时显示并记录。三模式一键切换，满足从评审演示到量化验收的全部场景。

2.4 验证体系

本设计的验证围绕一个中心思想展开：软件执行器与硬件流水线是同一份规格的两个实现。这一结构是差分测试的工程化 [15]——以同一输入驱动两个

独立实现、用输出差异判定缺陷；规格本身用穷举与独立实现钉死，两个实现之间用逐像素对比闭环。如图 9 所示，验证分四层。

规格层：逐像素规格对混合输出给出唯一确定的计算定义——六种方程的系数、除以 255 的舍入顺序、中间量位宽、裁剪的四区交集、色键的比较域、字节序全部定稿，其中舍入与位宽由全定义域穷举校验支撑（ALPHA 预乘路径 8,421,376 组逐值一致；快速舍入式与精确商全域等价）。命令语义另以 159 条带字面期望值的一致性向量校验，覆盖字段矩阵、检查优先级、错误码与失败副作用，向量工具自带六项故障注入自检。

单元层：引擎各模块在仿真中验证，AXI 从机模型带随机反压与可配延迟（测试台不得比真实从机更强）；每个判据都配有故障注入对照用例，注入故障后判据必须报错；语义类判据进一步以变异测试（Mutation Testing）方式系统性构造故障样本，未被检出的变异即为判据缺口。

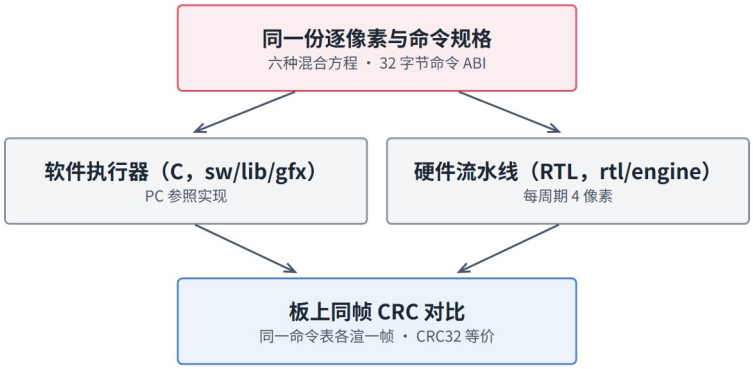


图 9　以同帧 CRC 为核心的四层验证体系

系统层：十一帧金标准用例覆盖六种方程、色键、裁剪与越界，板上同一命令表软硬各渲一帧、比较后缓冲 CRC32，全帧逐像素等价；性能版在真实游戏命令表上十组 CRC 全部相同，而故意跳过缓存失效的故障注入版十组全部不同——验证对比本身有效。条带路径以同一方法验收：生产构造层在含裁剪、跨桶、负坐标、源越界、别名与溢出的随机场景上与直通表做万张级逐字节差分，真实游戏整帧对独立软件金标准逐字节一致（可见区与行尾填充），

板上跨关卡 17 帧采样与 9,208 帧完整通关回放完成同帧 CRC 与性能计数对账；对桶归属、局部坐标与计数口径的故意变异均被检出。

性能层：硬件性能计数器与软件统计按同一统计定义采集，约束关系逐项对账；扫描 FIFO 水位、欠载计数与中断故障计数构成实时性的客观证据。全部性能数字以场数减切换数的帧率定义记录，与 HUD 显示一致。

第三部分　完成情况及性能参数

3.1 整体介绍

系统实物为一块 Ti60F225 开发板通过 HDMI 连接显示器，上位机经板载 USB 串口承担日志与输入转发；上电即从 Flash 自启动进入游戏，无需任何加载操作。图 10 为开发板与显示器实拍（厂商资料图），图 11 为系统实际运行屏摄：显示器上 MYRIAD 正以硬件后端运行，HUD 显示 HW FPS 61，桌面下方为开发板与上位机连线。

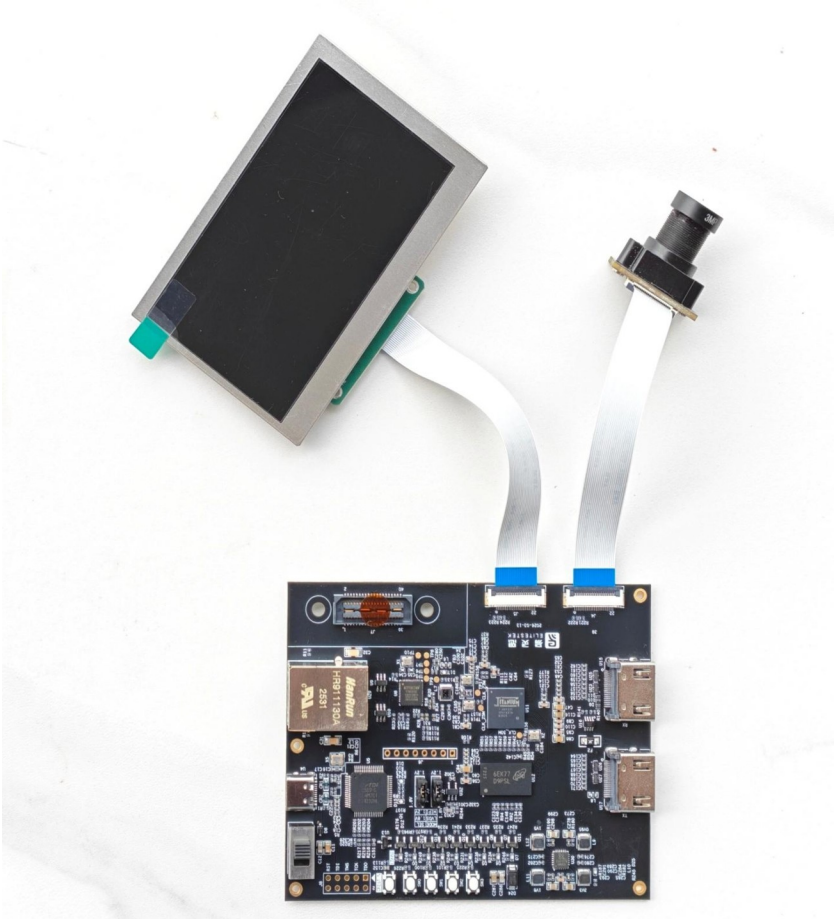


图 10　Ti60F225 开发板与显示器实拍（图片来源：易灵思《Ti60F225 DEMO 板软硬件设计说明》第 2 页）



图 11　系统运行屏摄：开发板经 HDMI 驱动显示器运行 MYRIAD，硬件后端 HUD 显示 61 FPS

3.2 工程成果

3.2.1 硬件成果

整机设计在 Efinity 2026.1 下编译，映射、布局布线、时序与比特流生成四步全部通过，无约束告警、无时序违例。表 3 给出整机资源占用及与平台基线的对比：当前完整构型（含硬件分桶与参数缓冲前端）在 98.8% 的 XLR 与 67.2% 的片上 RAM 预算下，仍保持四个时钟域全部正余量；三种执行模式共用同一像素通路，后续扩展以通路复用与内部位宽收窄换取空间。

表 3　整机资源占用与平台基线对比（实测）

资源	平台基线（SoC+总线+显示）	整机（含引擎）	占芯片总量
XLR 逻辑单元	16151	60064	98.8%
片上 RAM（10 Kbit/块）	65	172	67.2%
DSP	4	23	14.4%
core_clk 时序余量（setup）	+3.405 ns	+0.104 ns	无违例

整机中 12 个 DSP 与像素流水线的 12 个混合核一一对应，其余 DSP 承担分桶与引用前端的地址计算；当前片上 RAM 包含纹理缓存 16 块、条带缓冲 36 块、分桶计数与引用缓冲 8 块以及控制/命令缓冲。核心时钟 setup 余量 +0.104 ns、hold +0.022 ns，所有时钟域无负 slack。

3.2.2 软件成果

软件侧交付三部分：可玩的弹幕游戏 MYRIAD（含硬件渲染后端）、PC/板上双目标软件平台（同一份代码在两端逐行对账运行）、以及覆盖从素材打包到 Flash 自启动的完整交付链（应用区写入、独立读回校验、断电重启自启动均已在板上验证）。三个演示模式的画面见图 12–图 14；游戏逻辑与 HUD 统计在 PC 与板上运行同一份代码、逐行对账一致，板上硬件后端的实际运行见 3.1 节屏摄与 3.3 节实测数据。

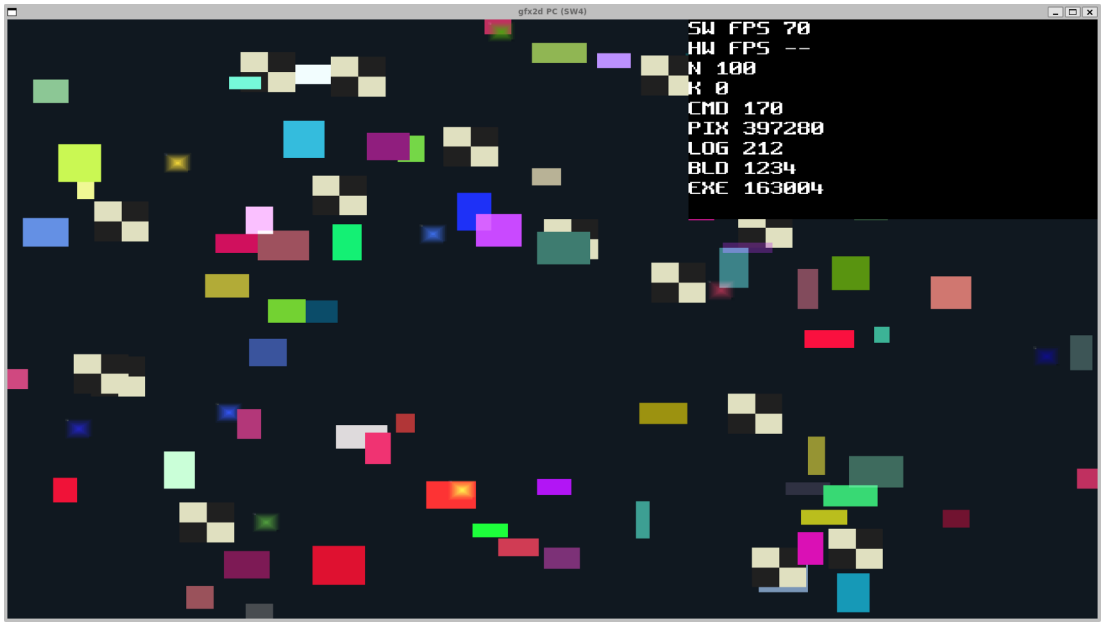


图 12　模式 1 基础演示：方块随机移动场景，HUD 同屏显示软件与硬件两路帧率（PC 解释器运行截图）

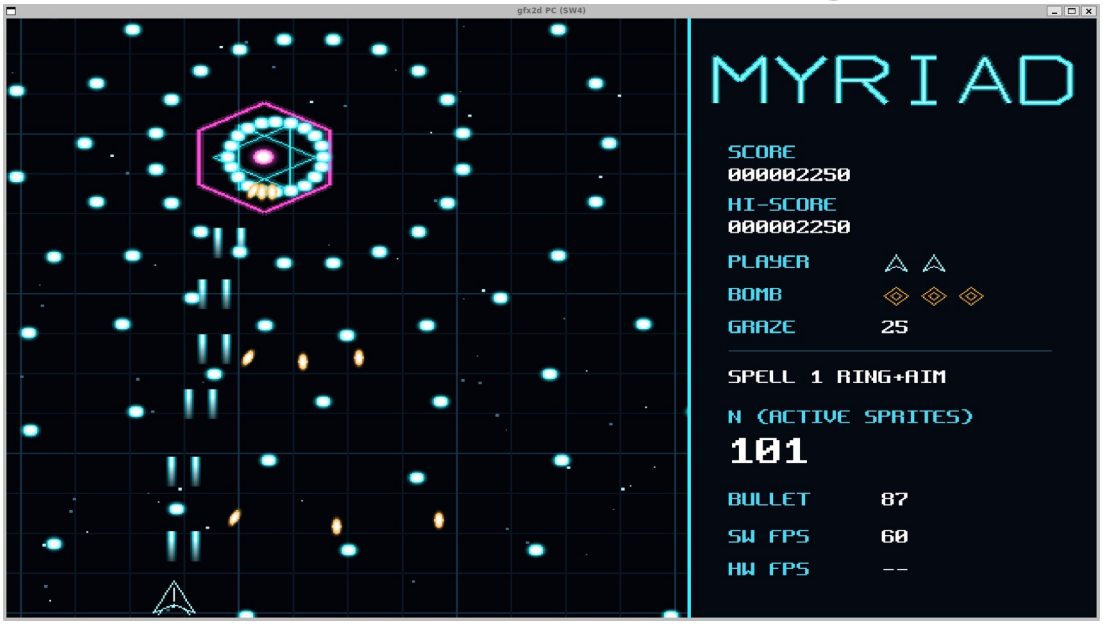


图 13　模式 2 弹幕关卡：Boss 符卡战画面，敌弹为预乘 Alpha 半透明 Sprite，色键去除背景（PC 解释器运行截图）

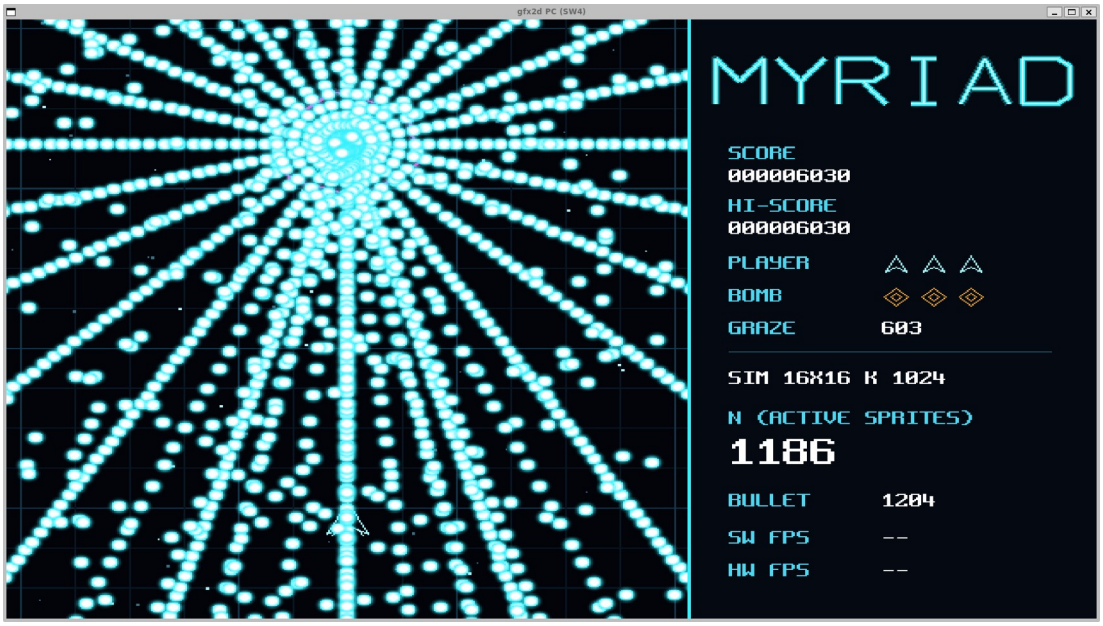


图 14　模式 3 压力测试：K1024 压力档下 1186 颗活动弹（PC 解释器运行截图，模拟时钟）

3.3 特性成果（量化指标）

本节数字全部为板上实测，测量口径统一如下：帧率取 HUD 同屏显示的硬件后端读数，并与 SWAP_COUNT、实际场计数对账；“稳定”判定要求连续 3600 场（60 秒）逐场零掉帧，且呈现帧数等于场数。N_{max} 采用倍增到首次

失败、再二分收敛的搜索，每档短测 120 场、候选档 3600 场确认。突发数、beat 数、等待周期等取自引擎内建性能计数器，并与“读 beat 数等于各读突发长度之和”这类约束逐项对账。资源占用与时序取自 Efinity 2026.1 的综合与静态时序报告。

3.3.1 软硬件渲染帧率对比

图 15 给出同一游戏、同一命令表在两个后端下的实测帧率（直通后端历史基线实测）。普通游戏短测：硬件后端首秒预热后稳定 60 FPS；纯软件后端约 2.5 FPS（单帧最长 38.45×10^6 周期，即约 384 ms），加速约 24 倍。压力负载（负载旋钮 1744，约 2030 颗活动弹，约为直通后端历史稳态极限档两倍的规模）：硬件后端 60 秒渲出 1839 帧、约 30.6 FPS，期间 FIFO 零欠载；同规模软件后端大量掉帧，场上存量降到约 650 颗。该对比直接回应赛题“在屏幕上实时对比显示纯软件渲染与硬件加速渲染的帧率差异”的要求：演示时屏幕 HUD 即同屏显示这两个数字。板上硬件后端的实际运行见图 16：屏摄中 HUD 显示 HW FPS 61、活动弹 292。

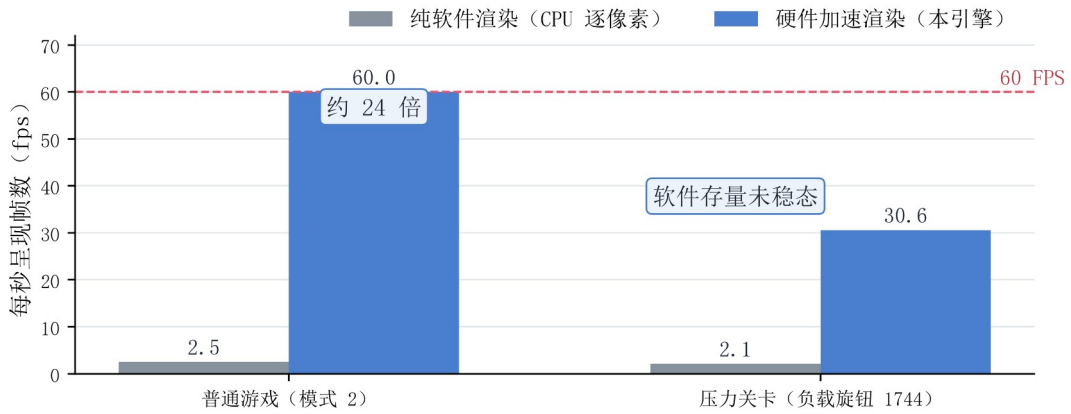


图 15　普通游戏与同负载压力档的实测帧率（直通后端历史基线；软件后端受 CPU 逐像素路径限制）

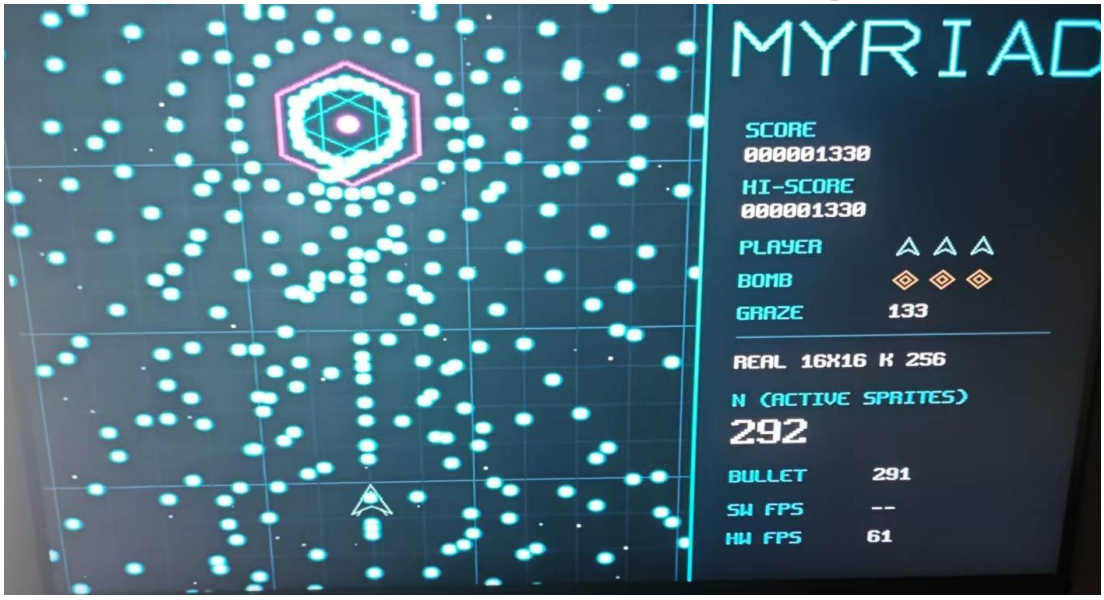


图 16　板上硬件后端运行屏摄：HUD 显示 HW FPS 61、活动弹 292（REAL 16X16 K 256 档）

3.3.2 同屏活动 Sprite 数量极限（N_{max}）

赛题高阶挑战要求“在开启硬件加速（Alpha 混合 + Color Keying）的情况下，挑战保持 60 FPS 稳定运行时的同屏活动元素数量极限”。本作品把该指标定义为可自动执行的判据：以真实游戏终符为负载，负载旋钮 K 控制每 125 帧发射的弹数，每颗弹渲染为一个对应尺寸的 Sprite；搜索采用倍增到首次失败、再二分收敛的策略，每档短测 120 场，候选档再以 3600 场（60 秒）确认，确认条件同时核对实际场计数与呈现帧数，要求 3600 场内逐场零掉帧——判据以场计数与呈现帧数对账，而不是只看 ft_max 是否低于 16.67 ms，局部掉帧因此无法掩盖。搜索轨迹见图 17（左 16×16、右 8×8）。直通后端历史基线的搜索最低水位 332 拍、欠载为零；条带后端三档同板实测最低水位为 331/323/321 拍；分桶后端搜索全程最低水位 322 拍，均无欠载。

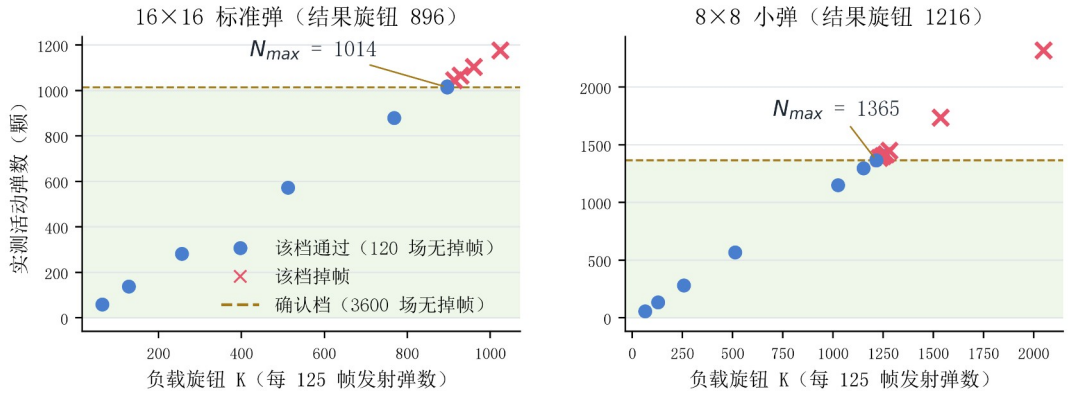


图 17　直通后端历史基线的 N_{max} 自动搜索轨迹（左：16×16；右：8×8）

直通后端历史基线的实测结果：16×16 标准弹 $N_{max} = 1014$ 颗（旋钮 896，确认档 3600/3600 场零掉帧，最长帧 1.638×10^6 周期）；8×8 小弹 $N_{max} = 1365$ 颗（旋钮 1216，确认档 3600/3600 零掉帧，最长帧 1.191×10^6 周期）。作为对照，同一负载下纯软件后端在初始 64 颗档即无法完成单场运行；硬件引擎将可玩弹幕密度提升了一个数量级以上。图 18 展示固定档 K896（约 1014 颗活动弹）连续 60 秒的逐秒帧时间与活动弹数：弹幕数量在约 4 秒内达到稳态，帧时间稳定在 16.3 ms 附近，接近 16.67 ms 预算，无累积劣化；60 秒内首秒为流水建立期（59/60），其后 59 个逐秒窗口全部 60/60。

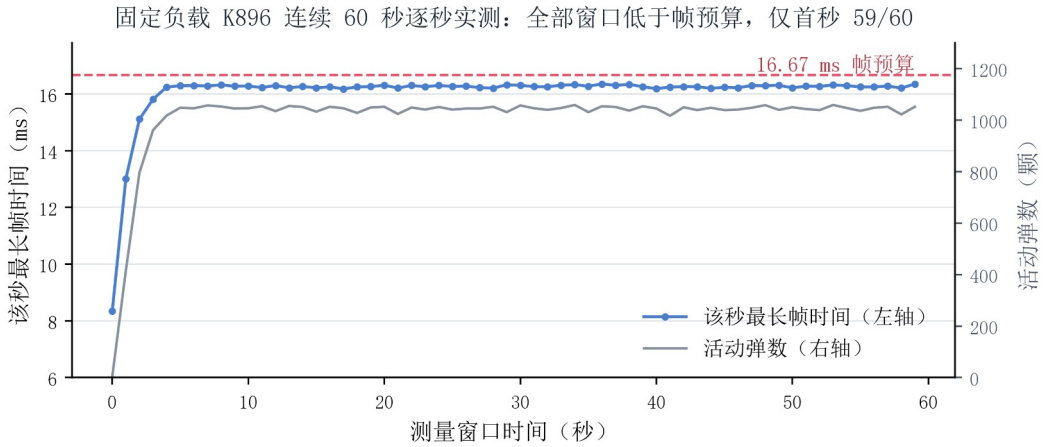


图 18　直通后端历史基线固定负载 K896 连续 60 秒的帧时间与活动弹数曲线

条带后端在同一份比特流上对照直通与条带两种执行，再用同一份条带程序比较两种缓存容量，数据见表 4。每种弹型均完成 3600 场零掉帧确认，所有搜索的扫描欠载和中断故障为 0：

表 4　条带后端的同板后端与缓存对照（实测）

构型与后端	16×16 N_{max} / K	8×8 N_{max} / K
4KB，直通（片上纹理）	1891 / 1648	2237 / 1984
4KB，条带	424 / 384	505 / 464
16KB 候选，条带	479 / 432	597 / 544

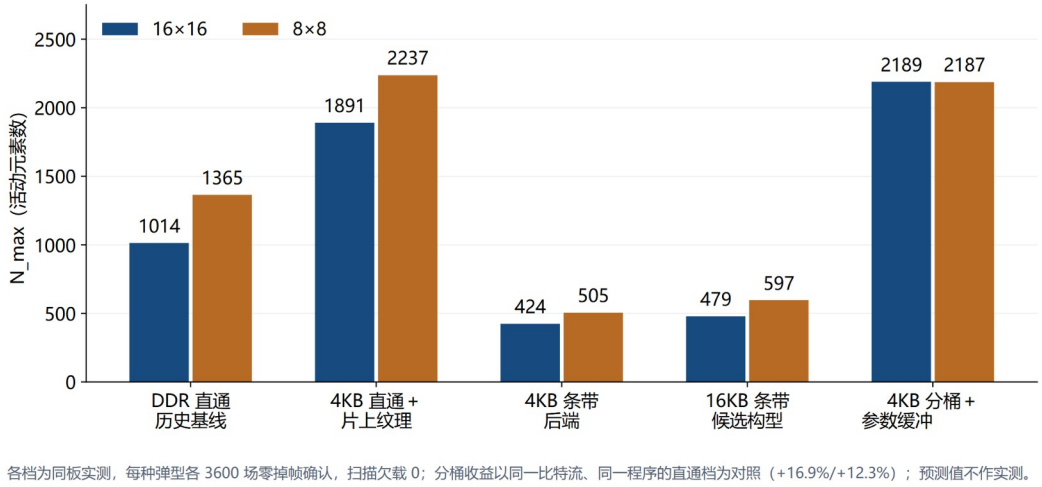


图 19　各后端与缓存构型的同板 N_{max} 对照；条带受 CPU 构造约束，分桶后端把构造搬入硬件后恢复增长

条带模式解决的是写方向：散落帧面的随机小写被合并为 30 次条带连续写，DDR 写事务因此收敛到可控量级。该组实测同时标定出下一个成本中心——CPU 构造约 1.3×10^{6} 周期、引擎约 0.6×10^{6} 周期，构造成为主项；16 KB 容量档把 N_{max} 再提高 13%/18%，说明片上容量对条带持续有效，而构造成本决定上限。直通基线同期引擎约 15.7 ms、CPU 约 9.3 ms，其瓶颈位于 DDR 事务固定成本。三组数据把优化目标从 DDR 侧推进到 CPU 侧，直接引出下面的硬件分桶。

针对这堵 CPU 构造墙，本作品新增硬件空间分桶（参数缓冲）后端：CPU 每帧只发布一份原始绘制与实例数组，分桶计算、跨带记录复制与引用表

构建全部移入硬件。分桶后端与直通后端以同一份比特流、同一份程序对照——程序同时携带两种后端的发布路径，运行时切换，消除程序差异对对照的干扰。每种弹型均以 3600 场（60 秒）确认零掉帧：

表 5　同一比特流上分桶与直通对照（实测）

后端	16×16 N_{max} / K	8×8 N_{max} / K
直通（片上纹理）	1872 / 1632	1947 / 1728
分桶（参数缓冲）	2189 / 1904	2187 / 1936

同板同程序对照下两种弹型 N_{max} 分别提高 16.9% 与 12.3%（表 5）。收益来源有两处：CPU 侧不再逐矩形复制命令与分桶，引擎侧按桶取用的引用表远小于复制的完整命令表——每帧读突发减少 70%、写突发减少 92%。对照所用的直通档携带分桶发布路径，其绝对 N_{max}（8×8 为 1947）与自身历史版本（2237，见表 6）不在同一程序口径；本节比较的是完全相同的程序、比特流与运行条件下的相对差距，16.9%/12.3% 因此可归因于分桶后端本身。

按 N_{max}×面积×60 换算，直通（片上纹理）为 29.0/8.59 Mpix/s，条带 4KB 为 6.51/1.94，16KB 候选为 7.36/2.29，分桶（参数缓冲）为 33.6/8.40；整机条件指标与像素阵列理论吞吐 400 Mpix/s 分别报告，避免把理论峰值当作实测。各后端与缓存构型的完整对照见表 4 与图 19。

3.3.3 赛题要求逐条对照

赛题七项基础要求与三项高阶要求在表 6 中逐条对照，每条给出实现方式与实测证据。

表 6　赛题要求逐条对照

赛题要求	本作品实现	实测证据
解析渲染指令（操作类型、源/目的地址、宽、高）	32 字节命令，五种操作码，表面表间接寻址	159 条语义向量全过；板上十一帧金标准 CRC 一致
图像块拷贝与纯色填充	BLIT / FILL，硬件裁剪与越界检查	同帧 CRC 逐像素等价
HDMI 输出不低于 640×480@60	TMDS 640×480@60，上电彩条基线	FRAME_COUNT 每秒 +60；displayTest 59.9 FPS
总线直接访问内存，突发与 FIFO	AXI4 128 位主机，突发 ≤192 拍参数化，多级 FIFO	突发上限 256 拍实测；FIFO 欠载恒为 0
C 驱动配置寄存器	APB 寄存器组 + 命令表 + DOORBELL + 中断	板上驱动全链路运行
双缓冲防撕裂	场消隐硬件锁存切换，掉帧有客观定义	SWAP_COUNT 与场数对账；切换仅发生于场消隐期
屏幕实时对比软/硬帧率	HUD 同屏显示两路帧率，一键切换后端	60 FPS 对 2.5 FPS（约 24 倍）
硬件 Alpha 混合（DSP）	12 个 DSP 混合核，预乘主路径 + 赛题原式对照路径	ALPHA 预乘 8,421,376 组穷举一致；整机 16 DSP
Color Keying	按源格式整字色键，命中跳写	KEY_SKIP 计数器对账；色键 Sprite 板上渲染正确
高负载游戏 Demo，60 FPS 下冲同屏 Sprite 极限	真实弹幕游戏 + 自动 N_{max} 搜索	分桶（参数缓冲）：2189 / 2187；直通（片上纹理）：1891 / 2237；各 3600 场零掉帧

3.3.4 可靠性证据

条带后端的搜索各有 3600 场零掉帧确认，扫描欠载和中断故障为 0；条带通关回放 9,208 帧与 PC 参考逐行一致，跨关卡同帧 CRC 与性能计数器对账通过。分桶后端以冻结源码重建全部测试程序并烧写最终比特流，九项板测

（正确性、既有后端回归、同程序性能对照、压力与完整回放）串行完成：分桶模式通关回放 9,208 帧逐帧一致；分桶正确性 74 组软硬件 CRC 相符、4 项错误注入全部检出；像素验证 153 场与组合场景 27 场全过；600 场压力稳态零掉帧。九项证据以同一份产物清单登记，指纹与部署记录逐项核对。九项板测经 JTAG 加载部署；Flash 写入、回读与自启动属此前已验收版本，证据见对应版本的验收记录。

第四部分　总结

4.1 可扩展之处

混合方程表、tint 与 INDEX8 字段为效果扩展保留了接口。提高分辨率需同步修改扫描时序、帧缓存布局与条带宽高，并重新核算带宽与场预算——这是分辨率提升的正确做法，而非只更换像素时钟。下一阶段的主攻方向已经锁定：引擎执行与 CPU 状态生成；条带搬运重叠（双条带缓冲的装载/写回与执行重叠）与 CPU 侧描述符化组帧是首选候选。16 KB/4 路对条带 N_{max} 的收益约 13%/18%，作为候选保留，采用与否由完整游戏与扫描实测决定。

4.2 心得体会

事务探针把架构取舍变成可复测的工程动作。三种后端各自消解一类瓶颈：直通消除重复读事务、条带消除随机写、分桶消除 CPU 构造，瓶颈因此逐级转移，最终落在引擎执行与 CPU 状态生成。这条链路本身就是本作品的方法论——先量瓶颈，再改结构，最后用同一比特流对照验证。

正确性由三条独立证据支撑：同规格软硬双实现、独立像素金标准、以及变异测试。每条交付结论都绑定串口日志、产物指纹与综合报告，可回溯到具体版本。

第五部分　参考文献

[1] 全国大学生嵌入式芯片与系统设计竞赛组委会, "2026 FPGA 创新设计赛道选题指南（易灵思）," 2026.

[2] Efinix Inc., "Titanium Ti60F225 FPGA Datasheet," v2.0, 2024.

[3] Efinix Inc., "Sapphire RISC-V SoC User Guide," v8.2, 2025.

[4] Efinix Inc., "DDR3 Soft Controller Core User Guide," v2.1, 2024.

[5] Efinix Inc., "Efinity Software User Guide," 2026.1, 2025.

[6] ARM Ltd., "AMBA AXI and ACE Protocol Specification," IHI 0022E, 2010.

[7] RISC-V Foundation, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," 2019.

[8] T. Porter and T. Duff, "Compositing digital images," in Proc. ACM SIGGRAPH, 1984, pp. 253–259.

[9] J. L. Bentley and A. C.-C. Yao, "An almost optimal algorithm for unbounded searching," Inf. Process. Lett., vol. 5, no. 3, pp. 82–87, 1976.

[10] STMicroelectronics, "AN4943: How to use the Chrom-ART accelerator (DMA2D) on STM32 MCUs," 2018.

[11] S. Molnar, M. Cox, D. Ellsworth, and H. Fuchs, "A sorting classification of parallel rendering," IEEE Comput. Graph. Appl., vol. 14, no. 4, pp. 23–32, 1994.

[12] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," in Proc. 27th Int. Symp. Computer Architecture (ISCA), 2000, pp. 128–138.

[13] K. Aoki and A. Yamawaki, "Development of a multi-sprite processing hardware engine combining high-level synthesis and on-chip FPGA memory," in Proc. 14th IIAE Int. Conf. Industrial Application Engineering (ICIAE), 2026, pp. 146–153.

[14] K. S. Ay and A. Dodan, "Hardware/software co-design of a 2D graphics system on FPGA," Int. J. Embedded Syst. Appl., vol. 3, no. 1, pp. 17–32, 2013.

[15] W. M. McKeeman, "Differential testing for software," Digital Technical Journal, vol. 10, no. 1, 1998.

[16] T. Akenine-Möller, E. Haines, and N. Hoffman, Real-Time Rendering, 4th ed. Boca Raton, FL, USA: CRC Press, 2018.

AI 工具使用说明

本作品使用 AI 辅助资料检索、代码实现与审查、文档校对和图表生成。构建、仿真、性能搜索与板卡测试由本机工具实际执行，原始日志和产物身份入库；生成内容经可复现判据核对，不以文字声明代替工程验证。